

Q.No.	Question	Expected Time to complete
	Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts Lab Objectives: ● Learn how to clean raw datasets using AI-assisted Python scripting. ● Apply preprocessing techniques such as handling missing values, encoding categorical data, and normalization. ● Automate repetitive data-cleaning tasks with AI-generated code. ● Understand how preprocessing impacts model performance. Task 1 Employee Dataset Cleaning Task: Use AI assistance to clean an employee information dataset. Instructions: ● Handle missing values in salary, designation, and experience ● Remove duplicate employee IDs ● Standardize text values (job titles in lowercase) ● Encode categorical features (designation, employment_type) Expected Output: A clean dataset suitable for HR analytics and prediction tasks. Prompt : <pre>Clean the employee dataset for HR analytics by filling missing values (median for salary, "unknown" for designation, mean for experience), removing duplicate employee IDs, standardizing job titles to lowercase, and encoding categorical features (label encoding for designation, one-hot encoding for employment_type). Return the cleaned dataset ready for predictive modeling.</pre>	

Code :

```
1 import pandas as pd
2 import numpy as np
3
4 # Create a sample DataFrame (replace this with your actual data loading)
5 data = {
6     'employee_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11],
7     'salary': [50000, 60000, np.nan, 70000, 80000, 90000, 100000, 110000, 120000, 130000, 140000],
8     'designation': ['Software Engineer', 'Data Scientist', 'Project Manager', 'Software Engineer', 'HR Specialist', 'Software Engineer', 'Data Scientist', 'HR Specialist', 'Software Engineer', 'Data Scientist', 'HR Specialist'],
9     'experience': [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0],
10    'employment_type': ['Full-time', 'Full-time', 'Contract', 'Full-time', 'Part-time', 'Full-time', 'Full-time', 'Full-time', 'Contract', 'Full-time', 'Part-time']
11}
12 df = pd.DataFrame(data)
13
14 print("Original Dataset Info:")
15 df.info()
16 print("Original Dataset Head:")
17 display(df.head())
```

```
18 # Fill missing 'salary' with the median
19 median_salary = df['salary'].median()
20 df['salary'].fillna(median_salary, inplace=True)
21
22 # Fill missing 'designation' with 'Unknown'
23 df['designation'].fillna('Unknown', inplace=True)
24
25 # Fill missing 'experience' with the mean
26 mean_experience = df['experience'].mean()
27 df['experience'].fillna(mean_experience, inplace=True)
28
29 print("Dataset after filling missing values:")
30 df.info()
31 display(df.head())
32
33 Dataset after filling missing values:
34 <class 'pandas.core.frame.DataFrame'>
35 RangeIndex: 12 entries, 0 to 11
36 Data columns (total 5 columns):
37 # Column Non-Null Count Dtype
38 ---
39 0 employee_id 12 non-null int64
40 1 salary 12 non-null float64
41 2 designation 12 non-null object
42 3 experience 12 non-null float64
43 4 employment_type 12 non-null object
44 dtypes: float64(2), int64(1), object(2)
```

```
df['salary'].fillna(median_salary, inplace=True)
#ipykernel.18972275480:39.py:18: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves like a copy. When you want to set a new value, you can use the .loc accessor.
for example, when doing 'df[col].method(value, inplace=True)', try using 'df.method([col], value, inplace=True)' or 'df[col] = df[col].method(value) inplace=True'

df['designation'].fillna('Unknown', inplace=True)
#ipykernel.18972275480:39.py:18: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves like a copy. When you want to set a new value, you can use the .loc accessor.
for example, when doing 'df[col].method(value, inplace=True)', try using 'df.method([col], value, inplace=True)' or 'df[col] = df[col].method(value) inplace=True'

df['experience'].fillna(mean_experience, inplace=True)
#ipykernel.18972275480:39.py:18: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves like a copy. When you want to set a new value, you can use the .loc accessor.
for example, when doing 'df[col].method(value, inplace=True)', try using 'df.method([col], value, inplace=True)' or 'df[col] = df[col].method(value) inplace=True'
```

```
Summary Statistics for the Cleaned Dataset
print(summary_statistics(df))
display(df.describe())
```

	employee_id	salary	experience	designation_encoded	employment_type_Contract	employment_type_Full-time	employment_type_Part-time
count	11.000000	11.000000	11.000000	11.000000	11.000000	11.000000	11.000000
mean	6.000000	63663.636364	6.272727	2.000000	0.181818	0.636364	0.181818
std	3.316625	4728.025722	2.370770	1.814088	0.404620	0.504025	0.404620
min	1.000000	50000.000000	2.000000	0.000000	0.000000	0.000000	0.000000
25%	3.000000	60000.000000	3.000000	0.500000	0.000000	0.000000	0.000000
50%	6.000000	61000.000000	6.000000	2.000000	0.000000	1.000000	0.000000
75%	8.000000	67000.000000	6.000000	3.000000	0.000000	1.000000	0.000000
max	11.000000	80000.000000	10.000000	5.000000	1.000000	1.000000	1.000000

Explanantion : employment_type_Contract,
employment_type_Full-time, employment_type_Part-time: These

are one-hot encoded binary variables (0 or 1). The mean for these columns represents the proportion of employees belonging to that employment type.

Task 2 – Student Performance Data Preprocessing

Task:Preprocess student performance data using AI assistance.

Instructions:

- Convert exam_date column into datetime format
- Extract new features (month, semester, day)
- Normalize total_marks
- Detect and handle outliers in marks using IQR or Z-score

Expected Output:

A transformed dataset with time-based academic features and normalized marks.

Prompt : :

Preprocess the student performance dataset by converting the exam_date column into datetime format, extracting new time-based features (month, semester, day), normalizing total_marks, and detecting/handling outliers in marks using either IQR or Z-score. Return the transformed dataset with academic time features and normalized marks, ready for analysis and prediction tasks.

Code :

```
import pandas as pd
import numpy as np

# Create a sample DataFrame for student performance
data = {
    'student_id': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115],
    'exam_date': pd.to_datetime(['2023-01-15', '2023-02-20', '2023-03-10', '2023-04-05', '2023-05-12',
                                '2023-09-01', '2023-10-10', '2023-11-15', '2023-12-01', '2023-01-25',
                                '2023-03-20', '2023-06-01', '2023-09-20', '2023-02-10', '2023-04-20']),
    'subject': ['Math', 'Science', 'Math', 'English', 'Science', 'Math', 'English', 'Science', 'Math', 'English',
               'Science', 'Math', 'English', 'Science', 'Math'],
    'total_marks': [85, 72, 91, 68, 78, 95, 76, 88, 82, 80, 75, 99, 58, 77, 105],
    'grade': ['A', 'B', 'A', 'C', 'B', 'A', 'B', 'A', 'B', 'A', 'A', 'B', 'C', 'B', 'A'],
    'attendance_rate': [0.95, 0.88, 0.99, 0.75, 0.92, 0.98, 0.80, 0.96, 0.70, 0.90, 0.85, 1.00, 0.65, 0.89, 0.91]
}

df_students = pd.DataFrame(data)

print("Original Student Dataset Info:")
df_students.info()
print("Original Student Dataset Head:")
display(df_students.head())

'''
Original Student Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 6 columns):
 #   Column              Non-Null Count  Dtype
'''
```

```
0 student_id 15 non-null int64
1 exam_date 15 non-null datetime64[ns]
2 subject 15 non-null object
3 total_marks 15 non-null int64
4 grade 15 non-null object
5 attendance_rate 15 non-null float64
dtypes: datetime64[ns](1), float64(1), int64(2), object(2)
memory usage: 832.0+ bytes

Original Student Dataset Head:
student_id exam_date subject total_marks grade attendance_rate
0 101 2023-01-15 Math 85 A 0.95
1 102 2023-02-20 Science 72 B 0.88
2 103 2023-03-10 Math 91 A 0.99
3 104 2023-04-05 English 68 C 0.75
4 105 2023-05-12 Science 78 B 0.92
```

```

Step 1: Convert exam_date to datetime and extract time-based features

We'll create month, semester, and day columns.

# Ensure 'exam_date' is datetime (if not already)
df_students['exam_date'] = pd.to_datetime(df_students['exam_date'])

# Extract month
df_students['exam_month'] = df_students['exam_date'].dt.month

# Extract day of the week (0-Monday, 6-Sunday)
df_students['exam_day_of_week'] = df_students['exam_date'].dt.dayofweek

# Define semesters (example: Jan-Jun = Semester 1, Jul-Dec = Semester 2)
def get_semester(month):
    if 1 <= month <= 6:
        return 1
    else:
        return 2

df_students['semester'] = df_students['exam_month'].apply(get_semester)

print("Dataset after extracting time-based features:")
df_students.info()
display(df_students.head())

```

```

Dataset after extracting time-based features:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 9 columns):
 #   Column                Non-Null Count  Dtype
---  --
 0   student_id            15 non-null    int64
 1   exam_date             15 non-null    datetime64[ns]
 2   subject               15 non-null    object
 3   total_marks           15 non-null    float64
 4   grade                 15 non-null    object
 5   attendance_rate        15 non-null    float64
 6   exam_month            15 non-null    int32
 7   exam_day_of_week      15 non-null    int32
 8   semester              15 non-null    int64
dtypes: datetime64[ns](1), float64(1), int32(2), int64(1), object(2)
memory usage: 1.1+ KB

```

	student_id	exam_date	subject	total_marks	grade	attendance_rate	exam_month	exam_day_of_week	semester
0	101	2023-01-15	Math	85	A	0.95	1	6	1
1	102	2023-02-20	Science	72	B	0.88	2	0	1
2	103	2023-03-10	Math	91	A	0.99	3	4	1
3	104	2023-04-05	English	68	C	0.75	4	2	1
4	105	2023-05-12	Science	78	B	0.92	5	4	1

```

print("Summary Statistics of the cleaned DataFrame")
display(df.describe())

```

```

Summary Statistics of the cleaned DataFrame:

```

	employee_id	salary	experience	designation_encoded	employment_type_contract	employment_type_full-time	employment_type_Part-time
count	11.000000	11.000000	11.000000	11.000000	11.000000	11.000000	11.000000
mean	4.000000	4081.806041	6.227273	2.000000	0.918182	0.045455	0.036364
std	3.316625	8726.203722	2.370070	1.014896	0.044220	0.044220	0.044220
min	1.000000	50000.000000	2.000000	0.000000	0.000000	0.000000	0.000000
25%	3.000000	58000.000000	3.500000	0.500000	0.000000	0.000000	0.000000
50%	6.000000	61000.000000	5.000000	2.000000	0.000000	1.000000	0.000000
75%	8.500000	67500.000000	6.500000	3.000000	0.000000	1.000000	0.000000
max	11.000000	80000.000000	10.000000	5.000000	1.000000	1.000000	1.000000

Explanation; Employee Dataset (df): Missing values were handled, duplicate employee IDs removed, job titles standardized, and categorical features encoded. Summary statistics were also provided.

Student Performance Dataset (df_students): exam_date was converted to datetime, time-based features extracted, total_marks normalized, and outliers in marks handled.

Task 3 – Student Health Data Preparation

Task: Clean and preprocess student health records using AI scripts.

Instructions:

- Handle missing values in height, weight, BMI
- Encode categorical features (medical_condition, fitness_status)
- Scale numerical features using standard scaling
- Split dataset into training and testing sets

Expected Output:

A preprocessed dataset ready for predictive health analysis.

Prompt :

Clean and preprocess the student health dataset by handling missing values in height, weight, and BMI, encoding categorical features such as medical_condition and fitness_status, scaling numerical features using standard scaling, and splitting the dataset into training and testing sets. Return the preprocessed dataset ready for predictive health analysis.

Code :

```
import pandas as pd
import numpy as np

# Create a sample DataFrame for student health data
data = {
    'student_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15],
    'age': [15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
    'gender': ['Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male'],
    'height_cm': [165, 170, 168, 172, 166, 175, 169, 171, 173, 167, 174, 169, 172, 170, 171],
    'weight_kg': [65, 68, 66, 70, 64, 72, 69, 71, 73, 67, 74, 69, 72, 70, 71],
    'bmi': [24.2, 23.8, 23.5, 23.1, 23.2, 23.6, 23.4, 23.3, 23.5, 23.2, 23.6, 23.5, 23.3, 23.4, 23.5],
    'medical_condition': ['None', 'None', 'None', 'Allergies', 'None', 'None', 'None', 'None', 'Allergies', 'None', 'None', 'Allergies', 'None', 'None', 'None'],
    'fitness_status': ['Good', 'Excellent', 'Average', 'Good', 'Average', 'Good', 'Excellent', 'Good', 'Average', 'Good', 'Average', 'Excellent', 'Good', 'Average', 'Excellent']
}

df_health = pd.DataFrame(data)

print("Original Student Health Dataset Info:")
df_health.info()
print("Non-null Student Health Dataset Head:")
display(df_health.head())
```

```
2 gender      15 non-null object
3 height_cm   15 non-null float64
4 weight_kg   15 non-null float64
5 bmi         15 non-null float64
6 medical_condition 15 non-null object
7 fitness_status 15 non-null object
dtypes: float64(3), int64(2), object(1)
memory usage: 1.2+ KB

Original Student Health Dataset Head:
student_id age gender height_cm weight_kg bmi medical_condition fitness_status
0          1   15   Male    165.0    65.0  24.2         None          Good
1          2   16  Female    170.0    68.0  23.8        Allergies      Excellent
2          3   15   Male    168.0    66.0  23.5         None          Average
3          4   17  Female    172.0    69.0  23.1        Allergies          Good
4          5   16   Male    166.0    70.0  25.5         None          Average
```

Step 1: Handle missing values

Step 1: Handle missing values

We will fill missing values in height_cm, weight_kg, and bmi with their respective medians.

```
df_health['height_cm'].fillna(df_health['height_cm'].median(), inplace=True)
df_health['weight_kg'].fillna(df_health['weight_kg'].median(), inplace=True)
df_health['bmi'].fillna(df_health['bmi'].median(), inplace=True)

print("Dataset after filling missing values:")
df_health.info()
display(df_health.head())

class 'pandas.core.frame.DataFrame'
RangeIndex: 15 entries, 0 to 14
Data columns (total 8 columns):
 #   column      Non-Null Count  Dtype
---  --
0   student_id  15 non-null         int64
1   age         15 non-null         int64
2   gender      15 non-null         object
3   height_cm   15 non-null         float64
4   weight_kg   15 non-null         float64
5   bmi         15 non-null         float64
6   medical_condition 15 non-null        object
7   fitness_status 15 non-null        object
```

```
df['height_weight_age'] = df['height'].weight.age
df['height_weight_age'].value_counts()
# value_counts() returns a Series with the counts of each unique value in the column. In this case, the unique values are the combinations of height, weight, and age.
# For example, when using df['method'(col, value), inplace=True], trying using df['method'(col, value), inplace=True] or df[col] = df[col].method(value) instead.
df['height_weight_age'] = df['height_weight_age'].weight.inplace
df['height_weight_age'].value_counts()
# value_counts() returns a Series with the counts of each unique value in the column. In this case, the unique values are the combinations of height, weight, and age.
# For example, when using df['method'(col, value), inplace=True], trying using df['method'(col, value), inplace=True] or df[col] = df[col].method(value) instead.
```

```

6 # If you have a specific target label, you may also define it here:
7 y = df.health[target_column]
8 x = df.health.drop(columns=["diabetes_1c", target_column])

9 # Split the data into training and testing sets (test_train, test_test)
10 X_train, X_test = train_test_split(x, test_size=0.2, random_state=42)

11 print("Shape of training features (X_train):", X_train.shape)
12 print("Shape of testing features (X_test):", X_test.shape)

13 print("\nFirst 5 rows of X_train:")
14 display(X_train.head())

15 # Shape of training features (X_train) (12, 14)
16 # Shape of testing features (X_test) (3, 14)

17 First 5 rows of X_train
18
19 age height_cm weight_kg bmi gender_female gender_male medical_condition_Allergies medical_condition_Asthma medical_condition_Diabetes med
20
21 0 1.165251 0.430379 0.134185 0.069900 1.0 0.0 0.0 0.0
22 1 0.274868 -1.996906 -0.906327 -0.868496 1.0 0.0 1.0 0.0
23 2 0.179137 0.179137 -0.027678 0.076206 0.0 1.0 0.0 0.0
24 3 0.274868 -1.638361 -0.277944 -0.326887 1.0 0.0 1.0 0.0
25 4 1.165251 0.280483 0.330391 0.320649 1.0 0.0 1.0 0.0

```

Explanation : Student Performance Dataset

(df_students): exam_date was converted to datetime, time-based features extracted, total_marks normalized, and outliers in marks handled.

Student Health Dataset (df_health): Missing values were handled, categorical features encoded, numerical features scaled, and the dataset split into training and testing sets.

Task 4: Real-Time Application: Ride-Sharing Data Cleaning

Scenario: A ride-sharing company has trip data with errors.

Requirements:

- Standardize pickup and drop locations
- Fill missing fare amounts with median values
- Remove duplicate ride entries
- Normalize driver ratings
- Generate a summary of data improvements

Expected Output:

A clean dataset ready for operational analysis.

Prompt :

Clean the ride-sharing trip dataset by standardizing pickup and drop locations, filling missing fare amounts with median values, removing duplicate ride entries, normalizing driver ratings, and generating a summary of data improvements. Return the cleaned dataset ready for operational analysis.

Code :

```
import pandas as pd
import numpy as np

# Create a sample DataFrame for ride-sharing trip data
data = {
    'ride_id': [1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009, 1010],
    'driver_id': [1, 2, 3, 4, 5, 1, 2, 3, 4, 5],
    'pickup_location': ['Downtown', 'Airport', 'Midtown', 'Downtown', 'Central Park', 'Downtown', 'Airport', 'Midtown', 'Downtown', 'Midtown'],
    'dropoff_location': ['Midtown', 'Midtown', 'Downtown', 'Airport', 'Brooklyn', 'Midtown', 'Midtown', 'Downtown', 'Downtown', 'Downtown'],
    'fare_amount': [25.50, 45.00, np.nan, 30.00, 35.00, 25.50, 50.00, np.nan, 25.50, 20.00],
    'driver_rating': [4.5, 3.8, 5.0, 4.2, 4.0, 4.5, 4.5, 4.5, 4.5, 4.0],
    'trip_distance_miles': [12.5, 15.0, 8.5, 7.0, 10.0, 9.5, 18.0, 6.0, 5.5, 6.0],
    'trip_duration_minutes': [18, 25, 20, 22, 25, 20, 30, 15, 20, 18]
}

df_rides = pd.DataFrame(data)

print("Original Ride-Sharing Dataset Info:")
df_rides.info()
print("Original Ride-Sharing Dataset Head:")
display(df_rides.head())

-- Original Ride-Sharing Dataset Info:
class 'pandas.core.frame.DataFrame'
Int64Index: 10 entries, 0 to 9
Data columns (total 8 columns):
 #   Column              Non-Null Count  Dtype
---  --
```

```

 #   Column              Non-Null Count  Dtype
---  --
0   ride_id             10 non-null    int64
1   driver_id           10 non-null    int64
2   pickup_location     10 non-null    object
3   dropoff_location    10 non-null    object
4   fare_amount         9 non-null     float64
5   driver_rating        10 non-null     float64
6   trip_distance_miles 10 non-null     float64
7   trip_duration_minutes 10 non-null     int64
dtypes: float64(3), int64(3), object(2)
memory usage: 836.0+ bytes

Original Ride-Sharing Dataset Head:
ride_id driver_id pickup_location dropoff_location fare_amount driver_rating trip_distance_miles trip_duration_minutes
0      1001         1      Downtown           Uptown         25.5          4.5             5.2             20
1      1002         2        Airport           Midtown         45.0          3.8            15.0             35
2      1003         3      Midtown           Downtown         NaN          5.0             3.5             15
3      1004         4      downtown           Airport         30.0          4.2             8.1             25
4      1005         5    Central Park           Brooklyn        35.0          4.0             7.0             22
```

```
We will clean these columns by removing leading/trailing whitespace and converting them to title case for consistency.

df_rides[['pickup_location', 'dropoff_location']]
df_rides[['pickup_location', 'dropoff_location']] = df_rides[['pickup_location', 'dropoff_location']].str.strip()

print("Dataset after standardizing locations:")
display(df_rides.head())

-- Dataset after standardizing locations:
ride_id driver_id pickup_location dropoff_location fare_amount driver_rating trip_distance_miles trip_duration_minutes
0      1001         1      Downtown           Uptown         25.5          4.5             5.2             20
1      1002         2        Airport           Midtown         45.0          3.8            15.0             35
2      1003         3      Midtown           Downtown         NaN          5.0             3.5             15
3      1004         4      Downtown           Airport         30.0          4.2             8.1             25
4      1005         5    Central Park           Brooklyn        35.0          4.0             7.0             22
```

Step 4: Normalize driver ratings

We will use Min-Max scaling to normalize 'driver_rating' to a range between 0 and 1.

```
from sklearn.preprocessing import MinMaxScaler

# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Reshape the 'driver_rating' column for scaling
df_rides[['driver_rating_normalized']] = scaler.fit_transform(df_rides[['driver_rating']])

print("Dataset after normalizing driver ratings:")
df_rides.info()
display(df_rides.head())

-- Dataset after normalizing driver ratings:
class 'pandas.core.frame.DataFrame'
Int64Index: 10 entries, 0 to 9
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  --
0   ride_id             10 non-null    int64
1   driver_id           10 non-null    int64
2   pickup_location     10 non-null    object
3   dropoff_location    10 non-null    object
```



```

6 trip_distance_miles 10 non-null float64
7 trip_duration_minutes 10 non-null int64
8 driver_rating_normalized 10 non-null float64
dtypes: float64(4), int64(3), object(2)
memory usage: 300.8+ bytes

ride_id driver_id pickup_location dropoff_location fare_amount driver_rating trip_distance_miles trip_duration_minutes driver_rating_normalized
0 1001 1 Downtown Uptown 25.5 4.5 5.2 20 0.666667
1 1002 2 Airport Midtown 45.0 3.8 15.0 35 0.200000
2 1003 3 Midtown Downtown 30.0 5.0 3.5 15 1.000000
3 1004 4 Downtown Airport 30.0 4.2 8.1 25 0.466667
4 1005 5 Central Park Brooklyn 35.0 4.0 7.0 22 0.333333

```

Summary of Data Improvements

```

Summary of Data Improvements

print("Overall Data Improvements:")
print(f"- Locations (pickup_location, dropoff_location) standardized to title case and trimmed whitespace.")
print(f"- Missing fare amounts filled with median value ({median_fare:2f}).")
print(f"- Initial_rides_count = rides_after_deduplication (duplicate ride entries removed).")
print(f"- Driver ratings normalized to a 0-1 scale, available in 'driver_rating_normalized'.")

print("Cleaned Dataset Ready for Operational Analysis:")
df_rides.info()
display(df_rides.head())

--- Overall Data Improvements:
- Locations (pickup_location, dropoff_location) standardized to title case and trimmed whitespace.
- Missing fare amounts filled with median value (30.00).
- 1 duplicate ride entries removed.
- Driver ratings normalized to a 0-1 scale, available in 'driver_rating_normalized'.

Cleaned Dataset Ready for Operational Analysis:
class 'pandas.core.frame.DataFrame'
Index: 10 entries, 0 to 10
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ride_id                10 non-null    int64
1   driver_id              10 non-null    int64
2   pickup_location         10 non-null    object

```

```

2 pickup_location 10 non-null object
3 dropoff_location 10 non-null object
4 fare_amount 10 non-null float64
5 driver_rating 10 non-null float64
6 trip_distance_miles 10 non-null float64
7 trip_duration_minutes 10 non-null int64
8 driver_rating_normalized 10 non-null float64
dtypes: float64(4), int64(3), object(2)
memory usage: 300.8+ bytes

ride_id driver_id pickup_location dropoff_location fare_amount driver_rating trip_distance_miles trip_duration_minutes driver_rating_normalized
0 1001 1 Downtown Uptown 25.5 4.5 5.2 20 0.666667
1 1002 2 Airport Midtown 45.0 3.8 15.0 35 0.200000
2 1003 3 Midtown Downtown 30.0 5.0 3.5 15 1.000000
3 1004 4 Downtown Airport 30.0 4.2 8.1 25 0.466667
4 1005 5 Central Park Brooklyn 35.0 4.0 7.0 22 0.333333

```

- Explanation : **ide_id**: Unique identifiers, ranging from 1001 to 1010.
- **driver_id**: IDs for drivers, ranging from 1 to 9.
- **fare_amount**: The average fare is 33.10, with a standard deviation of 8.18. Fares range from 25.50 to 50.00.
- **driver_rating**: The average driver rating is 4.33, with ratings spanning from 3.5 to 5.0.
- **trip_distance_miles**: The mean trip distance is 7.93 miles, with distances from 3.5 to 18.0 miles.
- **trip_duration_minutes**: The average trip duration is 23.2 minutes, with durations ranging from 15 to 40 minutes.
- **driver_rating_normalized**: This column shows the driver ratings scaled to a 0-1 range, with an average of 0.55.

